Software Requirements Specification

for

DecisionTwin AI: An Intelligent Decision Simulation and Risk Analytics Platform

Version 1.0 approved

Prepared by:

Preetish M 24BAD053

Saran Raja M 24BAD058

Lawrance Andrew Selvam B 24BAD047

Department of Artificial Intelligence and Data Science

Mepco Schlenk Engineering College, Sivakasi

Table of Contents

Table of Contents II
Revision History IV
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 4
2.3 User Classes and Characteristics 5
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements 7
3.1 User Interfaces 7
3.2 Hardware Interfaces 7
3.3 Software Interfaces 7
3.4 Communications Interfaces 8
4. System Features 9
4.1 User Profile & Digital Twin Creation 9
4.2 Decision Scenario & Option Management 10
4.3 Goals, Priorities & Constraints Configuration 10
4.4 ML-Based Predictive Analytics Engine 11
4.5 Monte Carlo Future Simulation Engine 12
4.6 Multi-Dimensional Risk Assessment 12
4.7 Goal Alignment & Decision Scoring 13
4.8 What-If Analysis Engine 14
4.9 Explainable AI Recommendation & Action Plan Report 15
5. Other Nonfunctional Requirements 15
5.1 Performance Requirements 15
5.2 Safety Requirements 16
5.3 Security Requirements 16
5.4 Software Quality Attributes 16

5.5 Business Rules 17
6. Other Requirements 18
6.1 Database Requirements 18
6.2 Legal, Compliance, and Internationalization Requirements 18
Appendix A: Glossary 19
Appendix B: Analysis Models 20
Appendix C: To Be Determined (TBD) List 21

Revision History

Name	Date	Reason For Changes	Version
		Initial SRS release for DecisionTwin AI (Mini Project III)	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for DecisionTwin AI, an intelligent decision simulation and risk analytics web platform. This document covers Version 1.0 of the complete system and is intended to serve as the primary reference for the design, development, and testing phases carried out as part of the Mini Project III (23AD553 – Software Engineering for AI Applications) coursework. The scope of this SRS covers the full end-to-end platform, including user profile and Digital Twin management, decision scenario modelling, machine-learning-based prediction, Monte Carlo future simulation, multi-dimensional risk assessment, What-If analysis, explainable AI reporting, and interactive dashboards.

1.2 Document Conventions

This document follows the IEEE 830-1998 recommended practice for software requirements specifications. Each functional requirement is uniquely tagged using the format REQ-<number> (e.g., REQ-1, REQ-2) and is numbered sequentially across the document for traceability. Requirement priority is expressed as High, Medium, or Low and is stated at the feature level; unless otherwise noted, a feature's priority is inherited by all of its detailed requirements. Section and heading numbers follow the standard IEEE SRS outline. The keywords "shall" and "must" indicate mandatory requirements, while "should" indicates a recommended but non-mandatory capability.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Student developer(s) implementing the system, who should read the entire document.
- Project guide / course faculty evaluating the Mini Project III deliverable, who may focus on Sections 1, 2, and 4.
- Testers, who should focus on Section 4 (System Features) and Section 5 (Other Nonfunctional Requirements) to derive test cases.
- Future end users or reviewers who wish to understand system capabilities, who should read Sections 1 and 2.

Readers are encouraged to begin with Section 1 (Introduction) and Section 2 (Overall Description) for context before proceeding to the detailed requirements in Sections 3–6.

1.4 Product Scope

DecisionTwin AI is an AI-powered web platform that helps individuals make complex, high-stakes life decisions such as higher education, career change, job selection, entrepreneurship, investment planning, and relocation through data-driven simulation rather than generic advice. Unlike conventional search engines or conversational AI chatbots,

DecisionTwin AI builds a Digital Twin of the user's current profile, evaluates every available option using trained machine learning models, simulates thousands of possible future outcomes using Monte Carlo simulation, and produces quantitative Decision, Risk, and Confidence scores for each alternative. The platform's goal is to make the reasoning behind each recommendation transparent through explainable AI, and to translate the chosen decision into a concrete, personalized action plan. This positions the product as a decision-support system and analytics platform, not a replacement for human judgment.

1.5 References

- IEEE Computer Society. IEEE Std 29148-2018: Systems and Software Engineering – Life Cycle Processes – Requirements Engineering. IEEE, 2018
- IEEE Computer Society. IEEE Std 830 – 1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE, 1998
- Roger S. Pressman and Bruce R. Maxim. Software Engineering: A Practitioner's Approach. 9th Edition, McGraw-Hill Education, 2019
- Ian Sommerville. Software Engineering. 10th Edition, Pearson, 2015.
- Karl E. Wiegers and Joy Beatty. Software Requirements. 3rd Edition, Microsoft Press, 2013

2. Overall Description

2.1 Product Perspective

DecisionTwin AI is a new, self-contained web application; it is not a follow-on to an existing product. It is composed of a React.js single-page front end, a Node.js/Express REST API layer, a MongoDB database, and a Java-based machine learning microservice that performs prediction, simulation, risk scoring, and explainability computations. The React client communicates with the Express API over HTTPS; the Express API in turn delegates prediction and simulation workloads to the Java ML microservice over an internal REST interface. The system may optionally integrate external market-data sources (job market trends, salary benchmarks, cost-of-living indices) to enrich its predictions; where live data is unavailable, curated reference datasets are used instead. Section 3.3 and Appendix B describe these interfaces and the overall data flow in more detail.

The diagram below illustrates the high-level system architecture and data flow across the React client, the Express API layer, the core backend services, the Java ML microservice, and the MongoDB database.

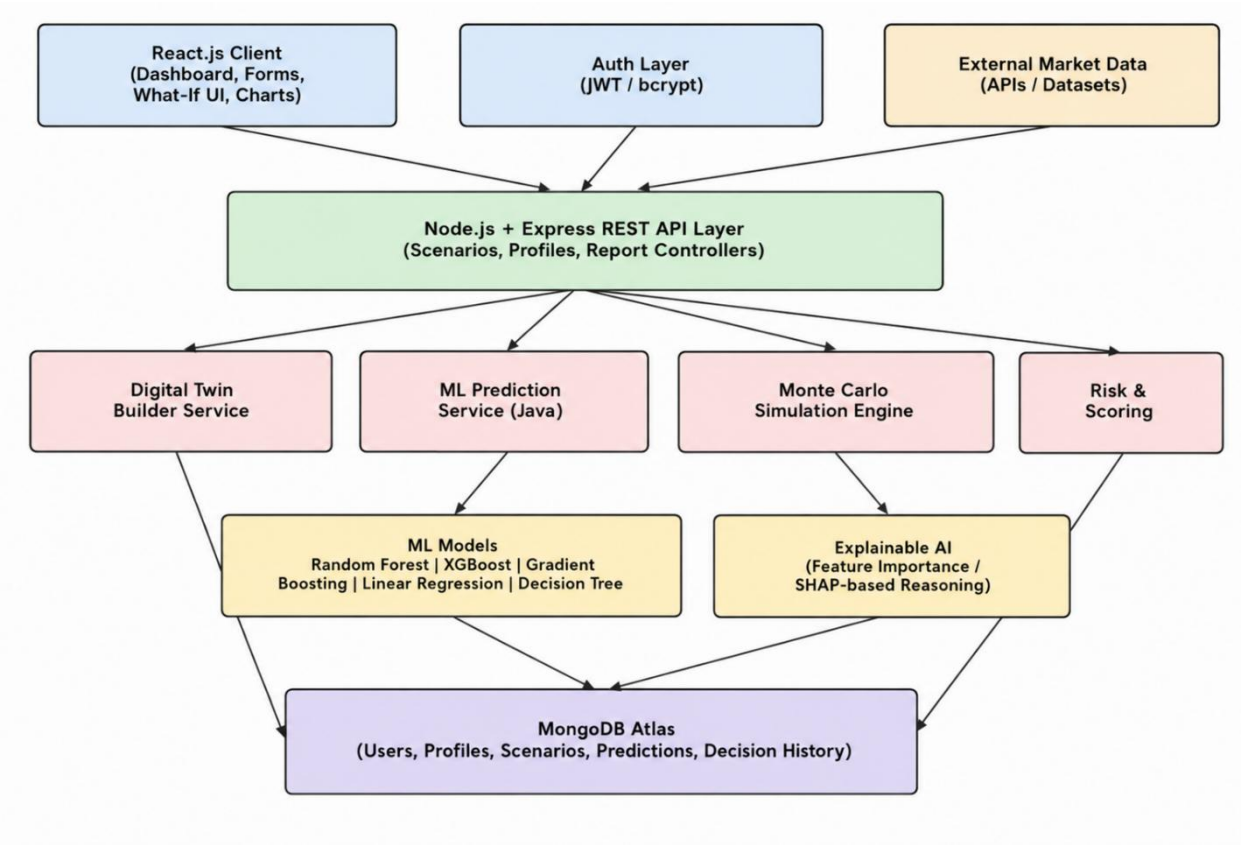


Fig 1. Architecture and data flow

Data flow summary:

- The React client sends authenticated requests to the Express REST API layer for profile, scenario, and report operations.

- The Express API delegates prediction, simulation, and explainability workloads to the Java ML microservice via internal REST calls.
- The ML microservice applies the appropriate model (Gradient Boosting Machine (GBM), Linear Regression, or Decision Tree) and returns predictions, which feed into the Monte Carlo Simulation Engine and Risk & Scoring modules.
- All persistent data - user profiles, scenarios, predictions, simulation results, and decision history is stored in MongoDB Atlas and retrieved by the API layer as needed.

2.2 Product Functions

The major functions the product are summarized below; details are provided in Section 4.

- Allow users to register, log in, and maintain a personal profile that forms the basis of their Digital Twin.
- Allow users to create and manage multiple independent decision scenarios, each with its own set of options.
- Collect user goals, priorities (weights), and constraints relevant to each decision scenario.
- Build and continuously update a Digital Twin representing the user's education, skills, financial status, experience, career stage, and risk preference.
- Predict quantitative outcomes (expected salary, ROI, employment probability, promotion probability, business success probability) for each option using trained ML models.
- Run Monte Carlo future simulations (e.g., 10,000 trials) to estimate best-case, average-case, and worst-case outcomes for each option.
- Compute multi-dimensional risk scores (financial, career, market, educational, personal, family) for each option.
- Compute a weighted Goal Alignment Score and an overall Decision Score/Confidence Score for each option.
- Allow users to perform What-If analysis by adjusting key parameters and instantly viewing recalculated outcomes.
- Generate explainable AI output describing why an option was ranked highest, its pros/cons, and its main risk contributors.
- Rank all options and generate a personalized, step-by-step action plan for the recommended option.
- Maintain a history of past decisions and use feedback to refine future recommendations.
- Present all of the above through interactive dashboards, comparison tables, timelines, and exportable reports.

2.3 User Classes and Characteristics

User Class	Characteristics
Registered End User	Individuals evaluating a personal or professional life decision (students, working professionals, entrepreneurs). Frequent use during an active decision; moderate technical expertise; primary consumers of all decision, simulation, and reporting features.
Guest / Unregistered Visitor	Explores platform capabilities using a limited demo scenario before registering. Cannot save decision history or access What-If analysis on saved scenarios.
System Administrator	Manages user accounts, monitors ML model/service health, updates reference market-data datasets, and reviews platform usage analytics. Highest privilege level.
Project Evaluator / Guide	Reviews the platform's functional completeness and correctness against academic project deliverables; typically interacts as a Registered End User with read access to sample scenarios.

The Registered End User is the most important user class to satisfy, since all core decision-support functionality is designed around this role.

2.4 Operating Environment

- Client: Any modern desktop or mobile web browser (Chrome, Firefox, Edge, Safari — latest two major versions) with JavaScript enabled and a minimum screen width of 360px for responsive layouts.
- Front end: React.js (v18+) single-page application, built and served as static assets.
- Application server: Node.js (v18 LTS+) with Express.js, deployed on a Linux-based cloud VM or container (e.g., Render, Railway, AWS EC2).
- Database: MongoDB (v6+), hosted on MongoDB Atlas or an equivalent managed/self-hosted instance.
- ML microservice: Java 3.10+ with Weka/Smile, Gradient Boosting Machine (GBM), and NumPy/SciPy, exposed as an internal REST service (Flask or FastAPI).
- The system must operate correctly over standard broadband and mobile data connections with typical latency.

2.5 Design and Implementation Constraints

- The system must be implemented using the MERN-based stack: MongoDB, Express.js, React.js, and Node.js, with a supporting Java ML microservice.

- Predictive analytics must use the specific algorithm family identified in the project scope: Gradient Boosting Machine (GBM), Gradient Boosting Machine (GBM), Gradient Boosting, Linear Regression, and Decision Trees.
- Future-outcome simulation must use a Monte Carlo simulation approach capable of generating at least 10,000 simulated trials per option within acceptable response time (see Section 5.1).
- All inter-service communication between the Express API and the Java ML microservice must occur over authenticated internal REST calls using JSON payloads.
- Where live external market data (job trends, salary benchmarks, inflation, currency rates) is not accessible, the system must fall back to a curated, versioned reference dataset stored in MongoDB.
- User-facing password storage must use salted hashing (bcrypt); session/authorization tokens must use JWT.

2.6 User Documentation

- An in-application onboarding walkthrough guiding first-time users through profile setup and their first decision scenario.
- Contextual help tooltips on all input forms (profile, goals, constraints, financial information).
- A user manual (PDF) covering account creation, scenario creation, interpreting Decision/Risk/Confidence scores, and using What-If analysis, delivered with the final project submission.
- An SRS and design document set (this document and its companions) delivered as academic project artifacts.

2.7 Assumptions and Dependencies

- It is assumed that users will provide reasonably accurate personal, financial, and goal information; the system does not independently verify user-submitted data.
- The system depends on third-party open-source ML libraries (Weka/Smile, Gradient Boosting Machine (GBM)) remaining available and compatible with the chosen Java version.
- Where real-time external market data feeds are not available or licensed, the project assumes the use of a static/curated reference dataset for market trends, average salaries, and cost of living, refreshed periodically by an administrator.
- It is assumed that MongoDB Atlas (or an equivalent hosting provider) remains available with the free/student tier sufficient for project demonstration purposes.
- The project assumes recommendations generated by the system are advisory in nature and are not a substitute for professional financial, legal, career, or medical counsel; this is communicated to users via in-app disclaimers (see Section 5.5).

3. External Interface Requirements

3.1 User Interfaces

DecisionTwin AI provides a responsive, React-based web interface. The following logical screens/components are required:

- Registration / Login screen with email and password fields, and form validation.
- Profile Setup wizard collecting the fields listed in Section 4.1, used to build the Digital Twin.
- Decision Scenario Creation wizard: decision category selection, option entry, goal and priority weighting sliders, and constraint entry.
- Decision Dashboard: overall Decision Score, Confidence Score, Risk Level, and top recommendation, using card and gauge-style visual components.
- Option Comparison view: a sortable, side-by-side table/chart of all options with Decision Score, Risk, and Confidence.
- Future Timeline view: an interactive, chronological milestone timeline for the recommended option.
- Risk Dashboard: separate visual indicators (e.g., radar chart) for financial, career, market, educational, personal, and family risk.
- What-If panel: editable parameter fields (salary, tuition, budget, loan amount, country, years of experience) with live-recalculated results.
- AI Recommendation Report screen: recommended option, explanation, pros/cons, risk summary, and suggested improvements, with an option to export as PDF.
- Action Plan view: a step-by-step, checkable roadmap tied to the recommended option.

All screens must follow a consistent design system (color palette, typography, spacing) and include standard elements such as a persistent navigation bar, a help/info icon, and clear error-message styling for invalid input. Detailed UI mockups are to be documented in a separate UI design specification.

3.2 Hardware Interfaces

DecisionTwin AI is a purely software-based web platform and does not interface directly with any specialized hardware.

3.3 Software Interfaces

Component	Purpose / Interaction
Node.js + Express.js API layer	Exposes REST endpoints for authentication, profile, scenario, goal/constraint, and report operations; validates input; orchestrates calls to MongoDB and the ML microservice.

Component	Purpose / Interaction
Java ML microservice	Receives feature data via internal REST calls; returns predictions from Gradient Boosting Machine (GBM), Gradient Boosting Machine (GBM), Gradient Boosting, Linear Regression and Decision Tree models; runs Monte Carlo simulations; computes Permutation Feature Importance-based explanations.

3.4 Communications Interfaces

- All client-server communication uses HTTPS with TLS 1.2 or higher.
- The React client and Express API exchange data as JSON over a RESTful API following standard HTTP verbs (GET, POST, PUT, DELETE).
- The Express API and the Java ML microservice communicate over an internal REST interface (JSON over HTTP/HTTPS), authenticated using a shared internal service key.
- Password and personally identifiable data must be encrypted in transit (TLS) and at rest (MongoDB encryption at rest, where supported by the hosting tier).

4. System Features

4.1 User Profile & Digital Twin Creation

Description and Priority

This feature allows a user to register, authenticate, and build a structured personal profile — the Digital Twin — capturing education, financial status, skills, experience, career stage, and risk tolerance. The Digital Twin acts as the baseline input for all downstream predictions, simulations, and risk calculations.

Priority: High

Stimulus/Response Sequences

- User registers with email/password; system validates and creates an account.
- User completes the profile wizard; system stores the data and marks the Digital Twin as "complete."
- User updates profile fields later; system recalculates any dependent open scenarios on next access.

Functional Requirements

REQ-1: The system shall allow a new user to register using email, password, and full name, and shall validate email format and password strength before account creation.

REQ-2: The system shall securely store passwords using salted hashing (bcrypt) and shall never store plaintext passwords.

REQ-3: The system shall allow a registered user to log in and shall issue a JWT access token valid for a configurable session duration.

REQ-4: The system shall provide a profile setup wizard capturing: age, highest education level, field of study, current occupation, years of experience, current salary/income, savings, debt/loans, monthly expenses, key skills, and self-declared risk tolerance (low/medium/high).

REQ-5: The system shall persist the completed profile as a structured Digital Twin document associated with the user's account in MongoDB.

REQ-6: The system shall allow the user to view and edit their profile at any time and shall timestamp the last update.

REQ-7: The system shall prevent creation of a new decision scenario until the user's Digital Twin profile is marked complete (see Section 5.5, Business Rule BR-1).

4.2 Decision Scenario & Option Management

Description and Priority

This feature lets a user create an independent decision scenario (e.g., "Which Master's program should I choose?") and define two or more options to be evaluated within that scenario.

Priority: High

Stimulus/Response Sequences

- User selects a decision category and creates a new scenario; system generates a scenario record linked to the user.
- User adds two or more options with option-specific attributes; system validates that each option has the minimum required fields.
- User archives or deletes a scenario; system removes it from active dashboards while retaining it in decision history.

Functional Requirements

REQ-8: The system shall allow a user to create a new decision scenario by selecting a decision category (e.g., education, career change, job offer, business/entrepreneurship, investment, relocation) from a predefined list.

REQ-9: The system shall allow a user to add, edit, and remove options within a scenario, with a minimum of two and a maximum of six options per scenario.

REQ-10: The system shall require category-specific attributes for each option (e.g., tuition cost and program duration for an education option; salary and role level for a job-offer option).

REQ-11: The system shall allow a user to save a scenario as a draft and resume editing it later.

REQ-12: The system shall allow a user to view a list of all their active and archived scenarios, sorted by last-modified date.

REQ-13: The system shall allow a user to delete a scenario and shall prompt for confirmation before permanent deletion.

4.3 Goals, Priorities & Constraints Configuration

Description and Priority

This feature captures what the user cares about (goals such as financial growth, job security, work-life balance, personal growth) and how much each goal matters (priority weight), along with hard constraints (e.g., maximum budget, must stay in current country) that options must respect.

Priority: High

Stimulus/Response Sequences

- User assigns a relative weight (0–100%) to each available goal for the scenario; system enforces that weights sum to 100%.
- User defines one or more hard constraints; system flags any option that violates a constraint.

Functional Requirements

REQ-14: The system shall present a predefined set of goal categories (financial growth, job security, work-life balance, personal growth/learning, family stability, location preference, health/well-being) for the user to weight.

REQ-15: The system shall provide a slider-based interface for assigning a percentage weight to each selected goal and shall enforce that all weights for a scenario sum to exactly 100%.

REQ-16: The system shall allow the user to define hard constraints such as maximum budget, maximum acceptable debt, minimum salary, or required location.

REQ-17: The system shall flag any option that violates a defined hard constraint and shall visually distinguish it in the comparison view (see Section 4.7).

REQ-18: The system shall allow the user to modify goal weights and constraints after initial **creation and** shall trigger score recalculation for all affected options.

4.4 ML-Based Predictive Analytics Engine

Description and Priority

This feature uses trained machine learning models to predict quantitative outcomes for each option, such as expected salary, return on investment (ROI), employment probability, promotion probability, or business success probability, depending on the decision category.

Priority: High

Stimulus/Response Sequences

- System sends the Digital Twin and option features to the ML microservice; the microservice selects the appropriate trained model(s) for the decision category.
- The ML microservice returns predicted outcome value(s) with an associated model confidence; system stores and displays the results.

Functional Requirements

REQ-19: The system shall generate a feature vector from the user's Digital Twin and each option's attributes and shall submit it to the ML microservice for prediction.

REQ-20: The system shall predict, at minimum, expected financial outcome (salary/ROI/profit) and a success/employment probability for each option.

REQ-21: The system shall return a model confidence indicator (e.g., prediction interval or probability score) alongside each predicted value.

REQ-22: The system shall cache prediction results for an option and shall only re-run predictions when the underlying profile, option data, or model version changes.

REQ-23: The system shall log model type, input features, and output values for each prediction for later evaluation and retraining.

4.5 Monte Carlo Future Simulation Engine

Description and Priority

This feature simulates a large number of possible future trajectories for each option (e.g., 10,000 trials) by sampling uncertain variables such as market conditions, salary growth rate, and inflation, producing best-case, average-case, and worst-case outcome distributions.

Priority: High

Stimulus/Response Sequences

- User requests a simulation for a scenario (automatically triggered after option/goal setup, or manually re-run); system runs Monte Carlo trials for each option.
- System aggregates trial results into percentile bands and displays them as a distribution/timeline chart.

Functional Requirements

REQ-24: The system shall run a Monte Carlo simulation of at least 10,000 trials per option, sampling key uncertain variables (e.g., annual salary growth, market demand shift, inflation rate) from configured probability distributions.

REQ-25: The system shall compute and display the 10th percentile (worst-case), 50th percentile (average-case), and 90th percentile (best-case) outcome for each option.

REQ-26: The system shall generate a projected outcome timeline (e.g., 1-year, 3-year, 5-year horizons) for the recommended option.

REQ-27: The system shall complete a full simulation run for a scenario within the performance bound defined in Section 5.1.

REQ-28: The system shall allow a user to manually re-trigger a simulation after modifying scenario inputs.

4.6 Multi-Dimensional Risk Assessment

Description and Priority

This feature computes a multi-dimensional risk score for each option across financial, career, market, educational, personal, and family dimensions, using the Digital Twin, option data, and simulation results as input.

Priority: High

Stimulus/Response Sequences

- System computes a risk sub-score for each risk dimension per option.
- System aggregates dimension scores into an overall risk level (Low/Medium/High) and displays it on the risk dashboard.

Functional Requirements

REQ-29: The system shall compute a financial risk sub-score based on debt-to-income ratio, savings buffer, and simulated worst-case outcome.

REQ-30: The system shall compute a career risk sub-score based on industry volatility, skill-gap, and predicted employment probability.

REQ-31: The system shall compute a market risk sub-score based on external market-data trends relevant to the decision category (see Section 3.3).

REQ-32: The system shall compute educational, personal, and family risk sub-scores where relevant to the decision category (e.g., relocation distance for family risk, program accreditation for educational risk).

REQ-33: The system shall aggregate all applicable sub-scores into a single overall Risk Level (Low, Medium, High) per option using a documented weighted formula.

REQ-34: The system shall display all risk dimensions for the recommended option on a dedicated Risk Dashboard (see Section 3.1).

4.7 Goal Alignment & Decision Scoring

Description and Priority

This feature combines predicted outcomes, simulation results, and risk scores with the user's goal weights (Section 4.3) to compute a single, comparable Decision Score and Confidence Score for every option.

Priority: High

Stimulus/Response Sequences

- System evaluates each option against every weighted goal to compute a Goal Alignment Score.
- System combines Goal Alignment Score, predicted outcome, and risk into a final Decision Score and ranks all options.

Functional Requirements

REQ-35: The system shall compute a Goal Alignment Score (0–100) for each option by evaluating how well its predicted attributes satisfy each user-weighted goal.

REQ-36: The system shall compute an overall Decision Score (0–100) for each option using a documented formula combining Goal Alignment Score, predicted outcome quality, and inverse risk level.

REQ-37: The system shall compute a Confidence Score reflecting the reliability of the Decision Score, derived from ML model confidence and simulation variance.

REQ-38: The system shall rank all options in a scenario by Decision Score, from highest to lowest, and shall identify the top-ranked option as the primary recommendation.

REQ-39: The system shall recalculate Goal Alignment and Decision Scores whenever goal weights, constraints, or option data change.

4.8 What-If Analysis Engine

Description and Priority

This feature allows a user to interactively adjust key parameters of an option (e.g., budget, expected salary, location, timeline) and see recalculated predictions, risk, and Decision Score without permanently altering the saved scenario.

Priority: Medium

Stimulus/Response Sequences

- User opens the What-If panel for an option and modifies one or more parameters.
- System recalculates predictions, risk, and Decision Score using the modified values and displays updated results alongside the original for comparison.

Functional Requirements

REQ-40: The system shall provide a What-If panel exposing the key adjustable parameters for the selected option's decision category.

REQ-41: The system shall recalculate predicted outcome, risk scores, and Decision Score based on modified What-If parameters within the performance bound defined in Section 5.1.

REQ-42: The system shall display the original (saved) result and the What-If result side by side for comparison.

REQ-43: The system shall allow the user to discard What-If changes or explicitly save them as an updated option.

REQ-44: The system shall limit What-If parameter adjustments to plausible ranges validated against the decision category's data constraints.

4.9 Explainable AI Recommendation & Action Plan Report

Description and Priority

This feature translates the raw scores and model outputs into a human-readable explanation of why an option is recommended, its pros and cons, its main risk contributors, and a concrete step-by-step action plan, exportable as a report.

Priority: High

Stimulus/Response Sequences

- System computes feature-importance/Permutation Feature Importance-based contributors for the top-ranked option's prediction and risk scores.
- System generates a structured explanation, pros/cons list, and action plan; user views it on-screen or exports it as a PDF report.

Functional Requirements

REQ-45: The system shall compute feature-importance or Permutation Feature Importance-based explanations identifying the top contributing factors behind each option's prediction and risk score.

REQ-46: The system shall generate a plain-language explanation of why the top-ranked option was recommended, referencing its strongest contributing factors.

REQ-47: The system shall generate a pros/cons list for the top two or three ranked options.

REQ-48: The system shall generate a step-by-step action plan (e.g., ordered milestones with suggested timelines) for the recommended option.

REQ-49: The system shall allow the user to export the recommendation report (explanation, scores, risk summary, action plan) as a downloadable PDF.

REQ-50: The system shall store each finalized recommendation in the user's decision history for future reference and model feedback.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall load the Decision Dashboard for a scenario with cached predictions within 2 seconds under normal network conditions.
- The system shall return ML prediction results for a new option within 5 seconds of request submission.
- The system shall complete a 10,000-trial Monte Carlo simulation for a single option within 8 seconds.
- The system shall recalculate and return updated What-If results within 3 seconds of a parameter change.

- The system shall support at least 100 concurrent active users during project demonstration without response times exceeding the bounds above by more than 50%.
- The MongoDB database shall be indexed on user ID, scenario ID, and decision category to keep typical query response times under 300 milliseconds.

5.2 Safety Requirements

- The system shall clearly label all outcome predictions, risk scores, and recommendations as advisory, algorithmically generated estimates, not guarantees.
- The system shall not delete a user's decision history or profile data without explicit, confirmed user action, in order to prevent accidental loss of decision-relevant records.
- The system shall maintain regular backups of the MongoDB database to guard against data loss.
- The system shall degrade gracefully if the ML microservice is unavailable, informing the user that predictions are temporarily unavailable rather than displaying incorrect or default values as if they were valid.

5.3 Security Requirements

- The system shall authenticate all API requests (other than registration/login and public demo endpoints) using a valid JWT access token.
- The system shall store all passwords using bcrypt salted hashing and shall never transmit or log plaintext passwords.
- The system shall enforce that a user can only access, modify, or delete their own profile, scenarios, and decision history (role-based / ownership-based access control).
- The system shall transmit all data between client, server, and ML microservice over HTTPS/TLS.
- The system shall validate and sanitize all user input on both client and server to prevent injection attacks (e.g., NoSQL injection, XSS).
- The system shall rate-limit authentication endpoints to mitigate brute-force login attempts.
- The system administrator role shall be the only role permitted to access aggregate/anonymized usage analytics; individual user financial and personal data shall not be exposed to other users.

5.4 Software Quality Attributes

Usability: The interface shall follow a consistent design system, provide contextual guidance during scenario creation, and require no more than three clicks to reach any core feature from the dashboard.

Reliability: Core prediction, simulation, and scoring features shall be covered by automated tests, and the system shall maintain at least 99% uptime during the demonstration/evaluation period.

Maintainability: The codebase shall follow a modular MERN architecture (separate routes, controllers, services, and models) and the ML microservice shall expose versioned model interfaces to simplify future model updates.

Scalability: The architecture shall allow horizontal scaling of the Express API and the Java ML microservice independently and shall support growth in the number of decision categories without core schema redesign.

Portability: The front end shall run on any evergreen browser; the backend and ML microservice shall run in standard Docker containers to simplify deployment across environments.

Availability: The system shall be available for demonstration and grading without scheduled downtime during the evaluation window defined by the course timeline.

5.5 Business Rules

- BR-1: A user must complete their Digital Twin profile before creating a decision scenario.
- BR-2: A decision scenario must contain at least two options before predictions, simulation, or scoring can be run.
- BR-3: Goal weights for a scenario must sum to exactly 100% before the scenario can be finalized.
- BR-4: All AI-generated recommendations must be accompanied by an explicit disclaimer that they are advisory and not professional financial, legal, medical, or career counsel.
- BR-5: A free-tier user may maintain a maximum of five active decision scenarios at a time; archiving a scenario frees a slot.
- BR-6: Only the owning user or a system administrator may view or export a given user's decision history and reports.

6. Other Requirements

6.1 Database Requirements

- The system shall use MongoDB collections for Users, Profiles/Digital Twins, Scenarios, Options, Goals/Constraints, Predictions, SimulationResults, RiskScores, and DecisionHistory, related via referenced ObjectIds.
- The system shall version the reference market-data dataset used as a fallback for external market data, recording the effective date of each dataset version.
- The system shall retain a user's full decision history indefinitely unless the user explicitly requests deletion, to support longitudinal recommendation refinement.

6.2 Legal, Compliance, and Internationalization Requirements

- The system shall present a disclaimer, accepted at registration, stating that outputs are AI-generated estimates and not professional advice (see Business Rule BR-4).
- The system shall allow a user to request export or deletion of their personal data, in line with standard data-protection principles.
- Currency and locale formatting shall default to Indian Rupees (INR) and Indian date formats, with the schema designed to support additional currencies/locales in a future release.
- Multi-language support (e.g., regional languages) is out of scope for Version 1.0 but is identified as a candidate for future work.

Appendix A: Glossary

Term	Definition
Digital Twin	A structured digital representation of a user's current education, financial status, skills, experience, and risk tolerance, used as the baseline input for all predictions and simulations.
Decision Score	A single 0–100 score combining Goal Alignment, predicted outcome quality, and risk, used to rank options within a scenario.
Confidence Score	A measure of how reliable a Decision Score is, derived from ML model confidence and Monte Carlo simulation variance.
Goal Alignment Score	A 0–100 score representing how well a given option satisfies the user's weighted personal goals.
Monte Carlo Simulation	A computational technique that runs a large number of randomized trials (sampling uncertain variables) to estimate a distribution of possible future outcomes.
Risk Level	An aggregated Low/Medium/High classification combining financial, career, market, educational, personal, and family risk sub-scores for an option.
What-If Analysis	An interactive feature allowing a user to temporarily modify option parameters and see recalculated predictions and scores without altering the saved scenario.
Explainable AI (XAI)	Techniques (e.g., Permutation Feature Importance, feature importance) used to make the reasoning behind an ML model's prediction interpretable to end users.
Action Plan	A generated, ordered set of milestones/steps helping a user execute the recommended option.
ROI	Return on Investment - the ratio of financial gain to the cost of a given option, used as a predicted outcome metric.

Appendix B: Analysis Models

The diagram below illustrates the ER diagram for our project:

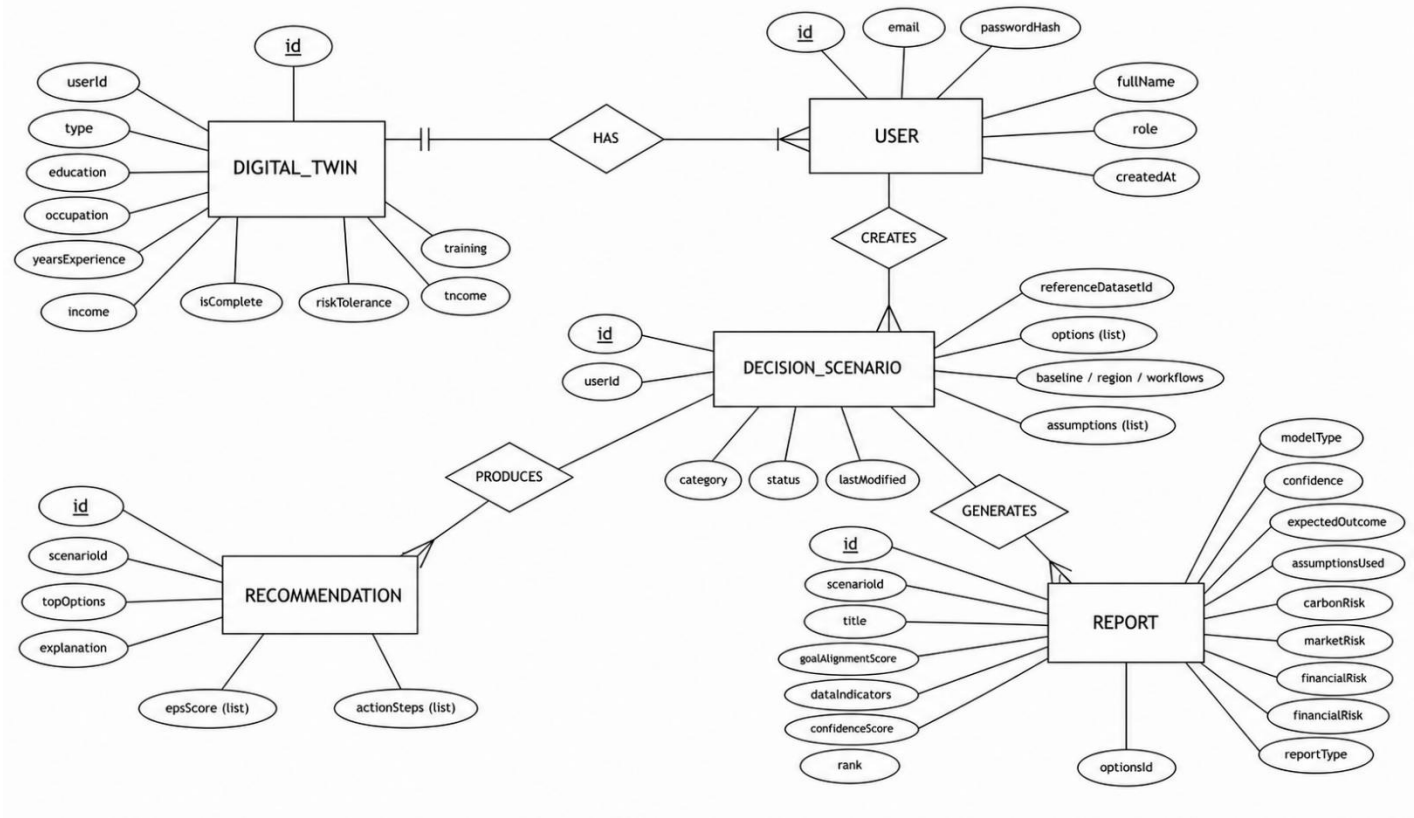


Fig 2. ER Diagram

Appendix C: To Be Determined (TBD) List

#	TBD Item
1	Final selection of external market-data provider(s) (job trends, salary benchmarks, cost of living) versus a fully curated static dataset.
2	Exact hyperparameters and training dataset(s) used for the Gradient Boosting Machine (GBM), Gradient Boosting Machine (GBM), Gradient Boosting, Linear Regression, and Decision Tree models per decision category.
3	Cloud hosting provider and CI/CD pipeline to be used for deployment of the Node.js API and Java ML microservice.
4	Whether real-time push updates (WebSocket) will be introduced for What-If analysis in a future release.
5	Scope and timeline for future multi-language / multi-currency support.